

COL1000: Introduction to Programming

Lecture 15

**Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi**

Structured Data Types in Python

A **structured data type** in Python is a way of organizing, grouping, and managing information.

In this course, we will study at the following **structured data types**:

- **Strings**
- **Tuples**
- **Lists**
- **Dictionaries**

Tuples

Tuples in Python

A **tuple** in python is an **ordered, immutable collection of elements** of arbitrary data types.

- Enclosed in parenthesis ()
- Each element separated by comma (,)

Example:

```
a = ()
```

```
b = ('Hello',)      # Comma is crucial (else treated as string)
```

```
c = ('Hello', 22)    # Tuple with a string and an integer
```

```
d = ('Hello', (22, 11)) # Tuple inside a tuple
```

Tuples : just like strings

Program

```
t = ("Hello", 5, "COL1000")  
  
print("Length:", len(t))  
  
print("First entry:", t[0])  
  
print("Last entry:", t[-1])  
  
print(t)
```

Output

```
Length: 3  
  
First entry: Hello  
  
Last entry: COL1000  
  
( 'Hello', 5, 'COL1000' )
```

Tuples : just like strings

Program

```
t = (1, 2, 3)

u = ("abc", "xyz")

print(t + u)

print(t*2 + u[0:1])

print(t[0:1] + u + t[2:3])
```

Output

```
(1, 2, 3, 'abc', 'xyz')
(1, 2, 3, 1, 2, 3, 'abc')
(1, 'abc', 'xyz', 3)
```

What will be the output of:

```
print(t[0] + u + t[2])
```

unsupported operand type(s)
for +: 'int' and 'tuple'

Tuples : just like strings

Program

```
t = (1, 2, 3)
u = ("abc", "xyz")

# concatenating tuples
w = t + u

print(w)

# iterating over tuple elements
for s in w:
    print(s)
```

Output

```
(1, 2, 3, 'abc', 'xyz')

1
2
3
abc
xyz
```


Tuples : just like strings

Program

```
t = (1, 2, 3)

u = ("abc", "xyz")

# tuple of tuples
w = (t, u)

print(w)

print(w[1][1])
```

Output

```
((1, 2, 3), ('abc', 'xyz'))

xyz
```


Computing Min/Max in Tuples

Program

```
def min_max(t):  
  
    if(len(t) == 0): return None  
  
    min = t[0]  
    max = t[0]  
  
    for s in t:  
        if (s > max): max = s  
        if (s < min): min = s  
  
    return (min, max)
```

```
z = (3, 5, -1, 4, 0)  
  
print(min_max(z))
```

Output

(-1, 5)

Function returns a tuple storing minimum and maximum

Summary of Tuples

Tuples : very similar to strings

- Strings are a sequence of characters,
- Tuples can be sequence of anything (including tuples).

Same set of operations as string.

Note: Tuples and strings are **non-mutable**.

```
t = "Hello"
t[0] = "h"           # will give an error

u = ((), ())
u[0] = ("Hello",)    # will give an error

u = ("Hello",) + u[1:2] # This will NOT give an error. Why?
```

COL1000: Introduction to Programming

Lecture 16

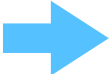
**Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi**

Mutable Sequence of Objects

Lists

Mutable vs Non-Mutables

What happens (internally) when we run the following code:



```
u = 2324  
v = 34411  
z = u + v
```

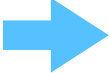
Memory:



u

Mutable vs Non-Mutables

What happens (internally) when we run the following code:



```
u = 2324  
v = 34411  
z = u + v
```

Memory:



Mutable vs Non-Mutables

What happens (internally) when we run the following code:



```
u = 2324  
v = 34411  
z = u + v
```

Memory:



Mutable vs Non-Mutable

For each of these assignments, Python does the following:

1. Finds a new memory location
2. Puts this value at that memory location
3. 'Remembers' the variable to memory location mapping

This memory location can be obtained using `id(var_name)`

```
u = 2324
v = 34411
z = u + v

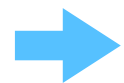
print(id(u))
print(id(v))
print(id(z))
```

Output

```
4388563576
4388563608
4388563672
```


Mutable vs Non-Mutables

Consider the following code, what do you expect the output to be?



```
u = 2324
print(id(u))

u = 34411
print(id(u))
```

Output

```
4388563576
4388563608
```

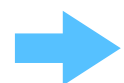
Memory:



u

Mutable vs Non-Mutable

Consider the following code, what do you expect the output to be?



```
u = 2324
print(id(u))

u = 34411
print(id(u))
```

Output

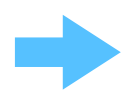
4388563576
4388563608



Integers (and all data-types we've seen so far) are non-mutable !

Mutable vs Non-Mutable

Consider the following code, what do you expect the output to be?



```
u = 13241
v = 22241

u = 33331
```

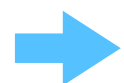
Memory:



Integers (and all data-types we've seen so far) are non-mutable !

Mutables vs Non-Mutables

Consider the following code, what do you expect the output to be?



```
u = 13241
v = 22241
u = 33331
```

Once an immutable object
loses its variable handle,
Python may free up
that memory slot :
garbage collection

Memory:



Integers (and all data-types we've seen so far) are non-mutable !

Lists: Mutable objects

Lists :

Enclosed in square brackets []

Each element separated by comma ,

```
t = []          # Empty list
```

```
u = ["Hello"]  # List with one string
```

```
# note the comma at the end is not needed for lists
```

```
v = ["Hello", 5] # List with one string and one integer
```

Lists: Mutable objects

Lists :

Enclosed in square brackets []

Each element separated by comma ,

Lists are mutable

```
t = [ "Hello" ]  
print(id(t))
```

```
t[0] = 5           # List with one integer  
print(id(t))      # id remains the same
```

Lists: Mutable objects

Lists :

Enclosed in square brackets []

Each element separated by comma ,

Lists are mutable

```
t = [ "Hello" ]  
print(id(t))
```

```
t = [ 5 ]      # List with one integer  
print(id(t))   # Does id remain same here?
```

Id changes every time we have an assignment

Lists: Mutable objects

Adding elements to lists

Adding elements to lists

Lists :

Can use **+** and ***** just like tuples and strings

```
t = [ "Hello" ]  
print(id(t))  
  
t = t + [ 5 ]      # List with one string and one integer  
print(id(t))      # Does id remain same here?
```

Id changes every time we have an assignment

Adding elements to lists

Lists :

can add element *e* to list *L*
using `L.append( e )`

```
t = ["hello"]  
print("id of t : ", id(t))  
  
t.append(5)  
print("t : ", t)  
  
print("id of t after append: ", id(t))
```

Output

```
id of t : 4332533696  
['hello', 5]  
id of t after append: 4332533696
```

Adding elements to lists

Lists :

can also add a list M to list L
using L.extend(M)

```
t = ["hello"]
print("id of t : ", id(t))

u = ["world"]
print("id of u : ", id(u))

t.extend(u)
print("t : ", t)
print("id of t after extend: ", id(t))
```

Output

```
id of t : 4332533696
['hello', 5]
id of t after append: 4332533696
```

Adding elements to lists

Lists :

can also add a list M to list L
using L.extend(M)

```
t = ["hello"]
print("id of t : ", id(t))

t.append(5)
print("t after append: ", t)

print("id of t after append: ", id(t))

u = ["world"]
print("id of u : ", id(u))

t.extend(u)
print("t after extend: ", t)
print("u = ", u)
print("id of u after extend : ", id(u))
print("id of t after extend : ", id(t))
```

Output

```
id of t : 4377065408
t after append: ['hello', 5]
id of t after append: 4377065408
id of u : 4376906304
t after extend: ['hello', 5, 'world']
u = ['world']
id of u after extend : 4376906304
id of t after extend : 4377065408
```

Adding elements to lists

Lists :

can also insert items at any position in L
using `L.insert( index, item )`

```
t = ["hello", "students"]  
print("id of t : ", id(t))  
  
t.insert(1, "ACOL100")  
print("t after insert: ", t)  
  
print("id of t after insert: ", id(t))
```

Output

```
id of t : 4377065408  
t after insert: ['hello', 'COL100', 'students']  
id of t after append: 4377065408
```

Removing elements from lists

`L.pop( )` : removes the last element in L

`L.pop( i )` : removes the element at position *i* in L

Both these methods modify the list, and also return the element removed

```
t = ["hello", "COL100", "students"]  
print("id of t : ", id(t))  
  
print(t.pop(1))  
print("t after pop: ", t)  
  
print("id of t after pop: ", id(t))
```

Output

```
id of t : 4377065408  
t after pop: ['hello', 'students']  
id of t after pop: 4377065408
```

Other operations on lists

Lots of built-in functions (methods) on lists.
The course objective is not to learn all the existing functions.
Make sure you understand how to use the basic ones.

Sorting a list

Given a list L ,
how to arrange them in increasing/decreasing order?

`L.sort( )` : mutates L, sorts elements in increasing order

`L.sort( reverse = True )` : mutates L, sorts elements in decreasing order

`sorted( L )` : sorts elements in increasing order, does not mutate L (returns a different list)

`sorted( L, reverse = True)` : sorts elements in decreasing order, does not mutate L

Summary of list operations

$L = [e_1, e_2, \dots, e_n]$: initialises list L with elements $e_1, e_2, \dots, e_n$

$\text{len}(L)$: returns the length of the list

$L + M$: concatenates lists L and M

$L * 2$: two copies of L , concatenated

$L.\text{append}(e)$: mutates L , appends e to L , returns None

$L.\text{extend}(M)$: mutates L , appends list M to L , returns None

$L.\text{insert}(i, e)$: mutates L , inserts e at position i , returns None

$L.\text{pop}()$: mutates L , removes the last element of L and returns it

$L.\text{pop}(i)$: mutates L , removes element at position i and returns it

$L.\text{remove}(e)$: mutates L , removes the first occurrence of element e

$L.\text{sort}()$: mutates L , sorts elements in increasing order, returns None

$L.\text{sort}(\text{reverse} = \text{True})$: mutates L , sorts elements in decreasing order, returns None

$\text{sorted}(L)$: sorts elements in increasing order, does not mutate L (returns a different list)

$\text{sorted}(\text{reverse} = \text{True})$: sorts elements in decreasing order, does not mutate L

Exercise: Matrix - Vector multiplication

Lists can be used for storing vectors, and list of lists for storing matrices

```
M = [[1, 2], [3, 4]]
```

```
v = [2, 1]
```

```
mult_output = [0, 0]
```

```
# complete code for matrix vector multiplication
```

```
for i in range(len(M)):
```

```
    for j in range(len(v)):
```

```
        mult_output[i] = mult_output[i] + M[i][j]*v[j]
```

```
print(mult_output)
```

Searching in sorted lists

For a list L , element e ,

$e \text{ in } L$: True if e is an element in L , False otherwise

Our Goal: implement a function `search_list( L, e )`

that takes as input a list L , element e ,

returns True if e is in L , False otherwise

(should not use any built-in functions)

Searching in sorted lists

```
def search_list(L, e):  
  
    left = 0  
    right = len(L) - 1  
  
    while (left <= right):  
        mid = int((left + right)/2)  
  
        if(L[mid] == e):  
            return True  
        elif (L[mid] < e):  
            left = mid + 1  
        else:  
            right = mid - 1  
  
    return False
```

What happens if we set left = mid ?

What happens if we set right = mid ?

COL1000: Introduction to Programming

Lecture 17

**Keerti, Naveen, Madhukar, Venkata
Department of CSE, IIT Delhi**

Aliases and Side Effects in Lists

Aliasing and side effects

Different names, same object

What is the output of the following code:

```
L = [1, 2, 4]
```

```
M = L
```

```
M[0] = 0
```

```
print(L)
```

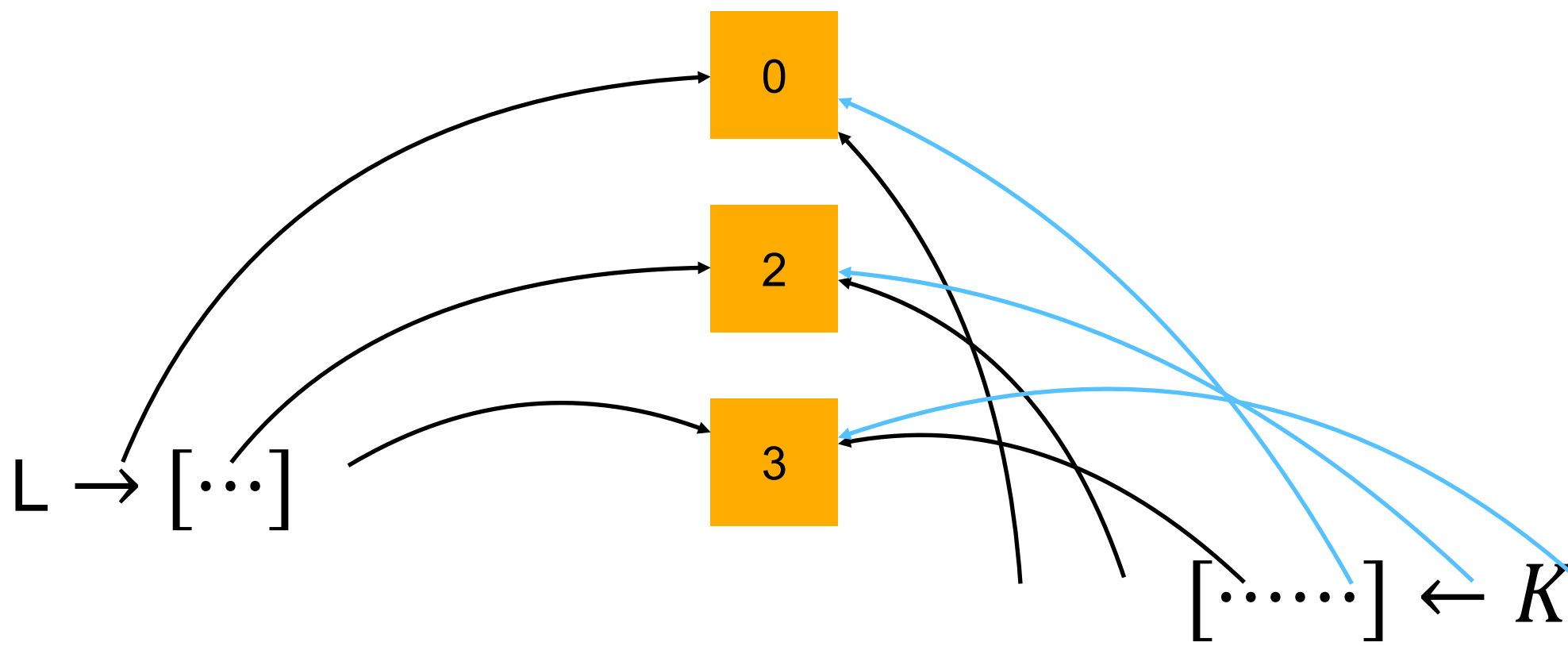

Aliasing and side effects

Different names, same object

What is the output of the following code:

```
L = [[1], [2], [3]]  
K = L*2  
K[0][0] = 0  
  
print("K = ", K)
```

```
K = [[0], [2], [3], [0], [2], [3]]
```



Aliasing and side effects

Different names, same object

What is the output of the following code:

```
L = [1, 2, 3]
K = L*2
K[0][0] = 0

print("K = ", K)
```

TypeError: 'int' object does not support item assignment

Aliasing and side effects

Different names, same object

What is the output of the following code:

```
L = [1, 2, 3]
K = L*2
K[0] = 0

print("K = ", K)
```

K = [0, 2, 3, 1, 2, 3]

Aliasing and side effects

Different names, same object

What is the output of the following code:

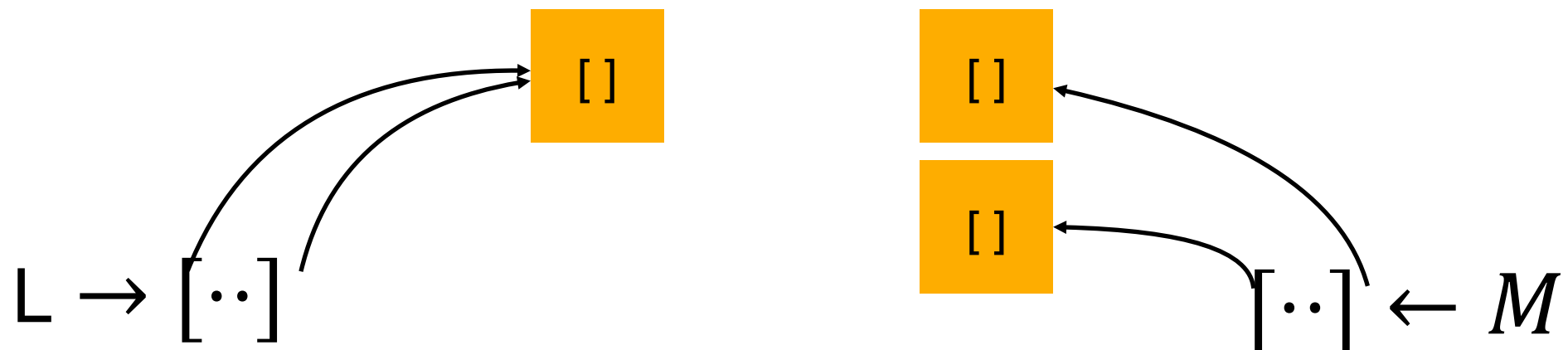
```
L = [[]]*2  
M = [], []
```

```
for i in range(2):  
    L[i].append(i)  
    M[i].append(i)
```

```
print("L=", L)  
print("M=", M)
```

Output:

```
L = [[0,1], [0,1]]  
M = [[0], [1]]
```



Functions and lists: More aliasing and side effects

What is the output of the following code:

```
def f(L):  
    L[0] = 0  
    return  
  
L = [2, 1, 4]  
f(L)  
print(L)
```

Output:

```
L = [0, 1, 4]
```

For mutable objects (like lists)
the function call passes a reference to the same list
(does not create a copy of L in the above example)

Functions and lists: More aliasing and side effects

What is the output of the following code:

```
def f(M):  
    M[0] = L[0] + M[0]  
    return  
  
L = [2, 1, 4]  
f(L)  
print(L)
```

Output:

```
L = [4, 1, 4]
```

Some more side effects

Different names, same object

What is the output of the following code (and why):

```
L = [1, 2, 3, 4]
M = [1, 2, 5, 6]

for e in L:
    if e in M:
        L.remove(e)

print("L = ", L)
print("M = ", M)
```

Output:

```
L = [2, 3, 4]
M = [1, 2, 5, 6]
```

**Alert: when iterating over a list,
be careful wrt mutating the list**

Copying a list

What is the output of the following code:

```
L = [2, 3, 4]
```

```
M = L
```

```
print(id(L))
```

```
print(id(M))
```

Output:

```
45254536
```

```
45254536
```

Alert: M and L both point to the same list

Copying a list

What is the output of the following code:

```
L = [2, 3, 4]
```

```
M = L[:]
```

```
print(id(L))
```

```
print(id(M))
```

Output:

```
45254536
```

```
42454632
```

`M = L[ : ]` creates a **shallow** copy of the list

Copying a list

What is the output of the following code:

```
L = [2, 3, 4]
M = L.copy()

M[0] = 0

print("L = ", L)

print("M = ", M)
```

Output:

```
L = [2, 3, 4]
M = [0, 3, 4]
```

`M = L.copy( )` also creates a **shallow** copy of the list

Copying a list

What is the output of the following code:

```
L = [[2], [3], [4]]  
M = L.copy()
```

```
M[0][0] = 0
```

```
print("L = ", L)
```

```
print("M = ", M)
```

Output:

```
L = [[0], [3], [4]]  
M = [[0], [3], [4]]
```

To create completely independent copies,
use `copy.deepcopy(L)`.
Also need to import copy library

Copying a list

```
import copy
L = [[2], [3], [4]]
M = copy.deepcopy(L)

M[0][0] = 0

print("L = ", L)

print("M = ", M)
```

Output:

```
L = [[2], [3], [4]]
M = [[0], [3], [4]]
```

Dictionaries

Dictionaries in Python

A Collection of (key, value) pairs, providing efficient look-up based on keys.

```
car = {  
    "brand": "Toyota",  
    "model": "Camry",  
    "year": 2020  
}
```

```
keys = car.keys()    # All keys  
values = car.values() # All values  
items = car.items()  # All (key, value) pairs
```

Dictionaries in Python

```
my_dict = {"Alice": 85, "Bob": 92, "Chen": 78}
```

```
# Iterating over the keys
```

```
for key in my_dict:
```

```
    print(key)
```

```
# Printing value for a specified key
```

```
print(my_dict["Alice"])
```

```
print(my_dict.get("Bob"))
```

```
# Adding new key-value pair
```

```
my_dict["John"] = 88
```

```
# Checking membership
```

```
if "John" in my_dict:
```

```
    print("Student's name is present")
```

Dictionaries to compute frequency

```
my_list = [9, 9, 1, 3, 1, 9]
```

```
frequency = {}
```

```
for item in my_list:
```

```
    if item in frequency:
```

```
        frequency[item] += 1
```

```
    else:
```

```
        frequency[item] = 1
```

```
print(frequency) # {9: 3, 1: 2, 3: 1}
```